

# **Pre-entrega TPE Tema 9 · Protección de Servicios con WAF para The Store**

**Grupo 2**

**Integrantes**

Mauro Vella

Enrique Castillo - 68321

Federico Inti Garcia Lauberer - 61374

## 1. Problemática y contexto

- The Store corre sobre Kind/Kubernetes y expone solo la UI vía `ingress-nginx`; los demás microservicios permanecen detrás de `ClusterIP` y no deberían ser accesibles desde el exterior.
- La auditoría pre-WAF realizada contra `http://localhost` confirmó 10 vectores explotables sin autenticación. Los más críticos son SSRF/path traversal vía `/proxy/\*\*`, IDOR de lectura en `/proxy/carts/{customerId}` y exposición de Spring Actuator.
- Hoy no hay rate limiting, scanner detection ni headers HTTP de seguridad. Esto deja abierta la puerta a enumeración, scraping, abuso automatizado, fuga de información operativa y reconocimiento previo a ataques más complejos.
- El problema es transversal a una arquitectura multi-lenguaje. Corregir individualmente en Java, Go y Node.js requiere cambios distribuidos; un WAF permite aplicar una política unificada en el punto de entrada.

## Hallazgos que motivan la solución

| ID | Hallazgo                                                             | Severidad | OWASP Top 10 |
|----|----------------------------------------------------------------------|-----------|--------------|
| H1 | SSRF + path traversal vía `/proxy/**`                                | Crítica   | A03 / A10    |
| H2 | IDOR de lectura en `/proxy/carts/{customerId}`                       | Crítica   | A01          |
| H3 | Spring Actuator expuesto (`info`, `health`, `metrics`, `prometheus`) | Alta      | A05          |
| H4 | Sin rate limit ni scanner detection                                  | Alta      | A04          |
| H5 | Headers de seguridad ausentes                                        | Media     | A05          |

## 2. Diseño de la solución

- Se propone desplegar `ModSecurity v3` con `OWASP Core Rule Set (CRS)` dentro del `ingress-nginx-controller` ya presente en el cluster Kind.
- La solución operará en dos etapas: `DetectionOnly` para tuning inicial y luego `On` para bloqueo efectivo, minimizando falsos positivos antes de la demo final.
- Además del CRS base, se incluirán reglas custom para bloquear `/actuator/\*\*` sensibles, negar traversal sobre `/proxy/\*\*`, limitar abuso automatizado y reforzar headers de seguridad.
- La principal ventaja es que no requiere cambios funcionales en los microservicios: el cambio queda acotado al Ingress, su ConfigMap y las reglas del WAF.

## Componentes a implementar

- ConfigMap de `ingress-nginx`: Activar `enable-modsecurity`, CRS y `SecRuleEngine` en modo `DetectionOnly`/`On`.
- OWASP CRS: Cubrir traversal, scanners y payloads genéricos de capa 7 sin tocar el código de negocio.
- Reglas custom: Bloquear `/actuator/\*\*`, traversal sobre `/proxy/\*\*` y reforzar guardrails contextuales.
- Anotaciones del Ingress: Aplicar `limit-rps`, `limit-connections` y `more\_set\_headers` para hardening perimetral.

## 3. Scope del POC y casos de uso

### Alcance comprometido

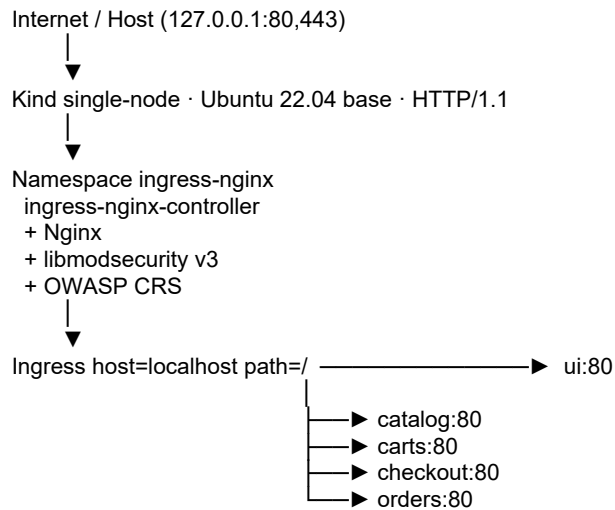
- Bloqueo de Actuator sensible: Requests a `/actuator/info`, `/actuator/metrics` y `/actuator/prometheus` deberán pasar de `200 OK` a `403 Forbidden`.
- Mitigación de SSRF/path traversal vía proxy: Payloads como `/proxy/catalog/../../actuator/prometheus` deberán quedar bloqueados por CRS 930xxx y/o regla custom específica.
- Detección de scanners conocidos: User-Agents de herramientas como Nikto o sqlmap deberán quedar registrados y bloqueados por reglas del CRS.
- Rate limiting en el punto de entrada: Se limitará el caudal por IP desde el Ingress para mitigar abuso automatizado y DoS de baja complejidad.
- Headers de seguridad HTTP: El Ingress agregará `HSTS`, `X-Frame-Options`, `X-Content-Type-Options`, `Content-Security-Policy`, `Referrer-Policy` y `Permissions-Policy`.

### 3. Scope del POC y casos de uso

#### Casos de uso para la demo

- Antes/después de exfiltración de métricas vía `/actuador/prometheus`.
- Antes/después de traversal vía `/proxy/catalog/../../actuador/info`.
- Request con User-Agent de Nikto antes/después del WAF.
- Burst de requests para mostrar ausencia/presencia de rate limiting.
- Comparación de headers de respuesta de la home antes/después.

#### 4. Diagrama de arquitectura



| Bloque             | CIDR          | Rol                                   |
|--------------------|---------------|---------------------------------------|
| Host loopback      | 127.0.0.1/32  | Usuario accede a The Store por 80/443 |
| Docker / nodo Kind | 172.18.0.0/16 | Red del contenedor del nodo           |
| Pods               | 10.244.0.0/16 | Comunicación entre pods               |
| Services           | 10.96.0.0/12  | ClusterIP internos                    |

- SO base del nodo: kindest/node sobre Ubuntu 22.04.
- Protocolos: HTTP/1.1 externo e interno; DNS interno de Kubernetes.
- Punto de inserción del WAF: ingress-nginx-controller en ingress-nginx.
- Cambios esperados: ConfigMap, reglas ModSecurity/CRS y anotaciones del Ingress principal.

## 5. Alternativas consideradas

- Shadow Daemon: Se descarta porque exige instrumentación más cercana a la aplicación y no encaja naturalmente con una arquitectura Java/Go/Node detrás de un Ingress compartido.
- OpenWAF: Fue considerado por figurar en el enunciado, pero se evita por menor madurez de ecosistema y menor predictibilidad para una demo académica.
- Coraza: Es técnicamente interesante y compatible con CRS, pero no figura en el tema asignado; se deja como evolución futura.

## 6. Plan breve de validación

- Usar como baseline la evidencia pre-WAF ya capturada en `pre-analysis/evidencias/`.
- Repetir los PoCs seleccionados con `DetectionOnly` y luego con bloqueo activo.
- Guardar snapshot post-WAF y comparar resultados pre/post con un diff simple.
- Adjuntar logs de ModSecurity para demostrar que el bloqueo provino del WAF.

## Métricas esperadas del antes/después

| Métrica                 | Pre-WAF     | Post-WAF esperado    |
|-------------------------|-------------|----------------------|
| /actuator/prometheus    | 200 OK      | 403 Forbidden        |
| Traversal vía /proxy/** | explotable  | bloqueado            |
| User-Agent Nikto/sqlmap | permitido   | detectado/bloqueado  |
| Rate limit              | no presente | activo en el Ingress |
| Headers de seguridad    | 0/6         | 6/6                  |

## 7. Riesgos y mitigaciones

- Falsos positivos del CRS: Arrancar en `DetectionOnly`, revisar logs y excluir reglas puntuales solo si afectan tráfico legítimo.
- Bloqueo insuficiente de vectores propios de la app: Complementar el CRS con reglas custom centradas en `/actuator/\*\*` y `/proxy/\*\*`.
- Dependencia del Ingress como punto único: Mantener cambios acotados y reproducibles en manifests para poder iterar rápido.

## 8. Cronograma resumido

- Semana 1: Validar pre-entrega y congelar alcance definitivo del POC.
- Semana 2: Desplegar ModSecurity + CRS sobre `ingress-nginx` en el cluster local.
- Semana 3: Implementar reglas custom y tunear falsos positivos.
- Semana 4: Generar baseline post-WAF, diff de resultados y material para la demo final.

## 9. Criterio de éxito

Se considerará cumplido el POC si los vectores hoy explotables en el punto de entrada HTTP quedan detectados o bloqueados por el WAF, sin romper el acceso normal a la home, catálogo, carrito y checkout. La entrega final incluirá demo funcional, código en GitHub y how-to de despliegue.